Abstract

In recent years e-commerce is rapidly growing. This growth has necessitated the development of an efficient, scalable and user-friendly systems for managing online stores. This project addresses the challenge of building a robust e-commerce solution capable to handle both administrative and customer functionalities. Traditional e-commerce platforms are often offer separate user and admin roles. This project addresses a more straightforward, unified e-commerce platform that allows user to also act as administrator, to manage products, and basic store functions efficiently. The main objective was to create an e-commerce platform that includes key functionalities like product catalog management, user account management and shopping cart operations. A dynamic update was given to the program after each action in order to ensure real-time synchronization. Additionally, it was given a focus on the implementation of an efficient sorting algorithm to organize products for better user experience. This system is designed under the assumption that users also serve as administrators simplifying role management. This project is implemented in Python based on object-oriented programming and it was rigorously tested and debugged to ensure reliability. It is designed to be easy to use and maintain, making it also be able to be expanded with additional features such as payment gateways, user authentication and sorting items. The platform also aims to bridge the gap between simplicity for small business owners and functionality to end-users

Key words: E-commerce, Product Management, User-friendly, Clothing, Object-Oriented Programming,

Table of Contents

1. Introduction *2*

1.1 Background of the application domain 2

1.2 Problem Statement 3

1.3 Importance of the project 3

2. Requirements Analysis *3*

2.1 Functional Requirements 3

2.2 Non-Functional Requirements 4

2.3 Constraint 5

3. System Design *5*

3.1 Architectural Design 5

3.2 Module Design 7

4. Implementation *10*

4.1 Tools and Technologies 10

4.2 Code Structure 11

4.3 Key Code Snippets 11

5. Results and Discussion *12*

5.1 Results 12

5.2 Discussion 14

5.3 Learnings 14

6. Conclusion and Future work *14*

7. Learning Objectives *14*

References *15*

1. Introduction

1.1 Background of the application domain

This project is based on how small e-commerce management systems work. This approach provides a session-based experience for users while maintaining a seamless flow between the customer and administrative functionalities. The system offers a small-scale e-commerce application for clothing items,

offering features like browsing, making an order, inventory management and discounts. With the evolution of e-shop environment, such systems are integral for allowing businesses to automate sales operations, enhance customer experience and efficiently manage their inventory.

1.2 Problem Statement

As trying to include both users' and admin's actions, the system effectively manages order by treating each session as independent. It also simplifies the process of managing both customer and administrative roles within the same environment. Interactions like addition, removal or updating the inventory from admin's point of view, become easier, simplifying the overall shopping experience ensuring users can manage their purchases effectively within the session. Dynamically capabilities have been integrated, making the system adjusts inventory levels in real-time. In this way, it provides users with accurate information on product availability throughout their shopping session. The menu system enhances user navigation, while it has been adjusted based on the customer's-admin's perspective.

1.3 Importance of the project

This system provides a simple and effective way for small to medium businesses to manage their operations efficiently. It achieved by management of orders, product catalog and discounts.

1.4 Objective(s) of the Project

Without needing a database, a functional system allows both customers and admin to manage interactions. From adding, updating, removing products - to manage customer orders effectively. It also ensures that the user experience is straightforward and adaptable to future expansions as will be discussed later on

2. Requirements Analysis

2.1 Functional Requirements

Product Management (Admin Level): New products can be added/removed from the main catalog with valid formats. Update of existing items such as **Price, Size** (S, M, L, XL, XXL) and **Stock**.

Admin Authentication: With Admin's code: "**GiorgosTzimas**" - access admin functionalities via password validation to ensure secure operations.

Product Display & Search (User Level): View all available products in the catalog with details such as "**Product ID**", "**Product Name**", "**Price**", "**Size**", and "**Stock**" and be able to search for specific products by "**Product ID**" or "**Product Name**" having implemented linear search.

Order Placement: Add products to a shopping cart if stock is available or display appropriate messages if not and/or remove previously added items from the shopping cart and update stock levels in the catalog.

Shopping Cart Management: View all items that are included in the shopping cart currently. Having the opportunity to apply a current valid discount code ("**WINTER15**") and calculate the discounted total. Proceed to checkout by entering customer details and generating an order confirmation receipt.

Main Menu Navigation: Unified interface to access all modules: Purchase functionality, Shopping cart management, Product display/search, Admin functions.

2.2 Non-Functional Requirements

Performance: The system can process user's and admin's commands within one second to ensure a smooth user experience where it can be handled thousands of products as dictionaries are manageable and efficient data structures because of their O(1) complexity

Usability: There are provided messages on each step with instructions for correcting input mistakes and there are simple menus in text format for an easy/friendly navigation.

Reliability: To avoid system problems, all processes validate inputs (valid sizes, discount code, etc.). Additionally, consistency is ensured by updating the stock for every product after adding or deleting items from the shopping basket.

Scalability: The system is able to support the integration of big datasets if it needs to, and it can handle it properly.

Security: To having access for making some updates in warehouse, admin-level activites require a unique password.

Maintainability: The codebase makes advantage of modular design principles (e.g., distict classes for ShoppingCart, Admin, etc.).

2.3 Constraint

Hardware/Software dependencies: The system is implemented in Python 3.12.4. For formatting the table of the "**product_catalog**", tabulate library has been used, as well as abc module (abstractmethod) to define a blueprint for other classes, making sure that specific behaviors are implemented by subclasses.

Time: In under one second the code is expected to process all available commands in our menu.

Budget: The system can be used without the use of external software/hardware which makes its design simple and cost-efficient

3. System Design

3.1 Architectural Design

This system's architectural design provides a high level overview of its components and interactions. Starting from block diagram (*Figure 1*):

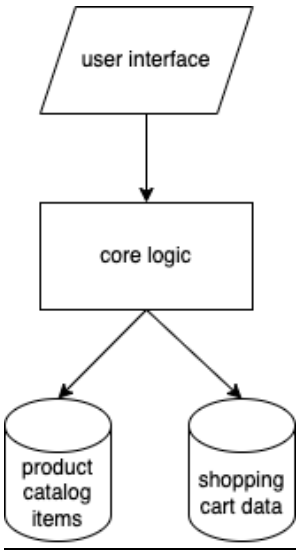

Figure 1

Definitions for block diagram:

- *Parallelogram: functional block (user interface layer)*: The role of this layer is the interactions between user and the system. It displays outputs based on specific inputs.

- *Rectangle: input/output operations (core logic layer):* This layer is the foundation of this system as it handles user requests, processes logic and coordinate operations between user interface and data layer.

- *Cylinder: data storage (product catalog items/shopping cart data layer):* The last layer consists of two categories while its role is storing and managing data.

Continuing to flowchart diagram (*Figure 2*):

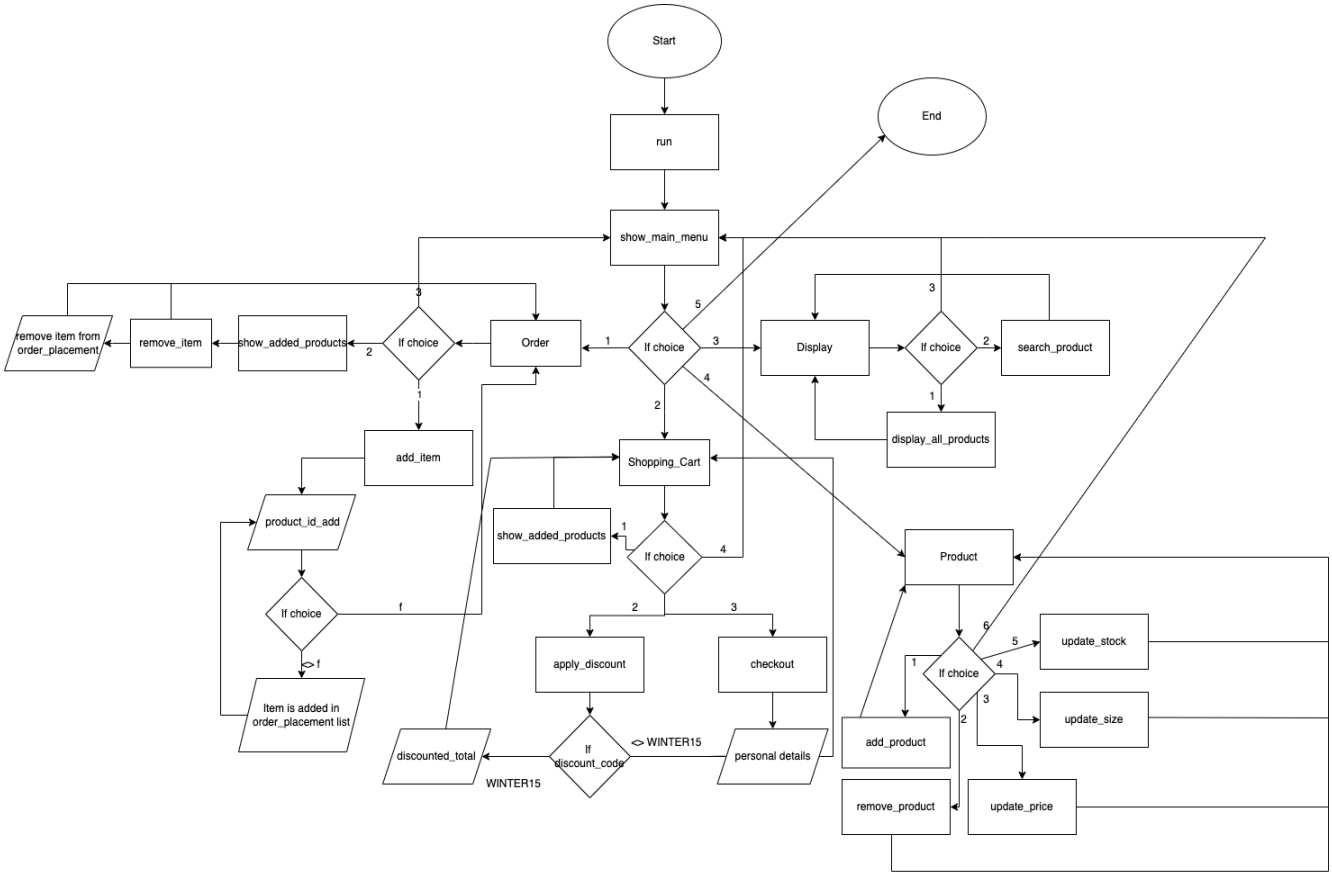

Figure 2

Definitions for flowchart diagram:

- *Oval*: Terminator
- *Rectangle*: Process
- *Diamond*: Decision
- *Parallelogram*: Data received by the system and data displayed or sent out by the system

3.2 Module Design

Modules:

⇒ *from abc import ABC, abstractmethod:* Module in Python's standard library. It provides tools for defining Abstract Base Classes (ABC).

⇒ *tabulate:* Third party module (not included in Python's standard library), used for creating a nicely formatted table from the list containing all products.

⇒ *import random:* Module in Python's standard library, used for generating random numbers and making random choices from sequences.

Class

⇒ *Menu*: It inherits from ABC, making it an abstract base class. It works as a blueprint for the other subclasses, where every subclass has to define a method (*show_menu* as we will analyse) ensuring consistency across different menu types. All the rest of the classes are inherited from *Menu* in order Python to dynamically select the appropriate implementation.

Method:

⇒ *show_menu:* As decorated with *@abstractmethod,* any subclass of *Menu* will be implemented by the *show_menu* method. The purpose of this is that calling *show_menu* in different subclasses, it ensures that each one will have its own unique functionality for displaying a menu (polymorphism) and consistent interface.

Class

⇒ *Order:* Childclass of *Menu*, it is designed to manage customer orders. It represents an order for a specific customer while it handles operations such as adding, removing and displaying products of a customer's order.

Attributes:

⇒ *order_id, customer_id*: Unique identifiers for the order and customer respectively

Methods:

⇒ *show_menu:* Interactive menu system for the customer in an order level environment.

⇒ *add_item:* Prompts the user to add in the basket displaying available items from the catalog, or to press "f" for going back. It validates the Product ID by checking the product_catalog and if valid and in stock it adds the product to the list *order_placement* and decreases the stock count in the catalog. In other case, it prints an appropriate message.

⇒ *remove_item:* Prompts the user to remove a specific item he has already added from his basket and the stock is updated in the catalog.

⇒ *show_added_products:* It displays all items included in user's cart.

Class

⇒ *Shopping_Cart:* It inherits Order and Menu combining their functionalities. It provides a user-friendly interface, handling customer orders while interacting with a specific menu.

Attributes:

⇒ It has access to *order_id* and *customer_id* as they have mentioned before

Methods:

⇒ *show_menu*: Interactive menu system in shopping cart environment.

⇒ *apply_discount:* Its usage is for applying discount to the current basket if user puts the specific current code correctly. In other case it raises a message of invalid code.

⇒ *display_order:* It displays all items of the cart.

⇒ *checkout:* It asks for detailed information for shipping the products. After each order, the *order_placement* list become empty until the next order.

In all above cases if the list *order_placement* (in which all items are added) is empty it returns a message.

Class

⇒ *Display:* Childclass of Menu and is designed for showing the products.

Methods:

- ⇒ *show_menu:* Interactive menu system for showing all or specific products available.
- ⇒ *display_all_products:* method for giving access and illustrate all available items at this specific point.
- ⇒ *search_product:* a linear searching algorithm, looping through each element from the start of the list *product_catalog* to its end until it finds and return the specific code of item the user has typed.

Class

- ⇒ *Product:* Childclass of Menu and is designed to allow the admin modify the *product_catalog*

- ⇒ **Methods:**
  - ⇒ *show_menu:* Interactive menu system in a warehouse level environment.
  - ⇒ *add_product:* System asks from the admin to type details of the product he wants to add in a specific format and the new items are added to the *product_catalog*
  - ⇒ *remove_product:* System asks from the admin to type a specific code and this item is removed entirely from the *product_catalog*
  - ⇒ *update_price:* It replaces the current price of a single product has on the catalog with the new one that admin input.
  - ⇒ *update_size:* It replaces the current size of a single product has on the catalog with the new one that admin input.
  - ⇒ *update_stock:* It replaces the current stock a single product has on the catalog with the new one that admin input.

  In all aforementioned methods there is a try-except application in order to handle incorrect inputs.

Functions:

- ⇒ *install_and_import_from:* It is created for installing and import the 3rd party module **tabulate** if it's not pre-installed in a different environment.
- ⇒ *generate_details:* It is created for generating random items which are added in the *product_list.*
- ⇒ *show_main_menu:* It displays an interactive main menu from which user can redirected to the option he chooses.

⇒ *run:* It is created for running the while e-commerce management having linked all classes together. It has also been assigned in a variable called "*begin*"

4. Implementation

4.1 Tools and Technologies

This e-commerce management system project has been built with pure Python. Its core is emphasized on OOP principles including some libraries, tools and methods.

- Tools

This program developed in **Jupyter Notebook** through **Anaconda** and it used for the interactivity it offers. Combining code, outputs and documentation in a single environment, it assisted for debugging, testing and presenting the outputs effectively.

- Libraries

Regarding Python's internal standard libraries, abc and random have been integrated into the program. abc's tools helps to define abstract base classes. It is used to define the *Menu* class, ensuring all subclasses will implement *show_menu* method, based on each class's requirements. random library helped for generating random operations in creating a list of items.
Regarding Python's external library, tabulate was used in order to give a table format to our list of products. There is no need of running *pip install* because in Jupyter Notebook through Anaconda, this library is already installed and included by default. Just in case to run on every environment, a different function was created to install it and run it successful without errors.

- Built-in methods

The following built-in methods are included in the system with their functionalities:
.clear(): for resetting the contents of shopping cart after successful checkout.
.strip(): for removing whitespace from user input to ensure input validation
.split(): to divide user's input string into separate fields based on the delimeter "," (comma)
.lower(): for converting string to lower-case ensuring case-insensitive input handling
.append(): for adding elements to lists "*product_list, search_results, product_catalog, order_placement"*

4.2 Code Structure

The structure of this e-commerce system in Jupyter Notebook is organized as follows:

1. The notebook starts by importing the necessary libraries: abc, random, tabulate, critical for generating randomness in lists, creating a consistent structure for the code and visualize catalog in a pretty way.
2. The notebook includes definitions of the essential classes *Menu, Display, Order, Shopping_cart.*
3. The central role in user experience plays the functions show_menu, display_all_products, add_item and remove_item where prompts the user to browse based on his preferences.
4. Instances of various components are initialized.
   Order Instances: represent user orders initialized with order ID and customer ID
   Display class: Manages the display and interaction with the menu.
   Shopping_Cart: Combines functionalities from Order and Menu.
5. Additional functions are included for inspecting and verifying the state of instance.
6. Finally, by showcasing interactions between classes and instances, a simulation of the system is demonstrating.

4.3 Key Code Snippets

We will turn our focus on key classes and functions that show the most important aspects of the code.

1. Display Class: Handles the display of the products and allows users to search and view details. It showcases effective management of user interaction with product data.
2. Product Class: It manages product creation, updates and removal. It demonstrates the implementation of efficient product data management and inventory control
3. Order Class: It handles the order placement and product stock verification during checkout. It highlights the order flow in connection with inventory updates.
4. Shopping_Cart Class: It combines order management with discount functionality and checkout process. It illustrates the management of handling both cart functionality and pricing logic.
5. Global function: generate_details(). It generates initial product data for creating our inventory.

5. Results and Discussion

5.1 Results

Starting the program

```
Welcome to the store!

Fixed 15% discount to our all products with the code: WINTER1

Press:
1. For Purchase
2. For Shopping Cart
3. To see our products
4. To enter as Admin
5. To Exit

Choose an option: 1
Press:
1. To add a Product Code
2. To remove a Product Code you want
3. To go back to main menu

Choose an option 1

Loading...

+--------------+---------------------------+---------+------
| Product ID   | Product Name              |   Price | Size
+==============+===========================+=========+======
| T533         | T-shirts 1                |     365 | XXL
```

Adding products and applying discount

```
+--------------+---------------------------+---------+--------+---------+
| U3956        | Underwear 49              |     561 | XL     |      11 |
+--------------+---------------------------+---------+--------+---------+
Please type the product you desire or press 'f' to finish:  U6548
The product U6548 has successfully been added.
Please type the product you desire or press 'f' to finish:  U3956
The product U3956 has successfully been added.
Please type the product you desire or press 'f' to finish:  f
Finished adding products.
Press:
1. To add a Product Code
2. To remove a Product Code you want
3. To go back to main menu

Choose an option 3
Back to main menu
Welcome to the store!

Fixed 15% discount to our all products with the code: WINTER15

Press:
1. For Purchase
2. For Shopping Cart
3. To see our products
4. To enter as Admin
5. To Exit

Choose an option: 2
Press:
1. My Shopping Cart
2. To apply discount
3. To Checkout
4. To go back to main menu

Choose one of the following options 2
Do you wanna apply a discount code?
yes/no yes
Enter the discount code:  WINTER15
Discount applied! Your total after discount is: 852.55 DKK.
```

Finishing the order

```
Press:
1. My Shopping Cart
2. To apply discount
3. To Checkout
4. To go back to main menu

Choose one of the following options 3

Proceeding to checkout...

Please enter your personal details:
Enter your name:  Giorgos Tzimas
Enter your email:  GioTz@test.dk
Enter your phone number:  +45 12 34 56 78

--- Order Confirmation ---
Customer Name: Giorgos Tzimas
Email: GioTz@test.dk
Phone: +45 12 34 56 78

Order placed successfully! Thank you for your purchase. We have sent you an email of your purchase information. We hope to see you again!
```

Searching for one of the products we have already purchased through searching method - and inspecting that the stock has been reduced from the inventory

```
Welcome to the store!

Fixed 15% discount to our all products with the code: WINTER15

Press:
1. For Purchase
2. For Shopping Cart
3. To see our products
4. To enter as Admin
5. To Exit

Choose an option: 3

Press 1 to display all products
Press 2 to search for a product
Press 3 to go back to the main menu
Choose an option:  2

Search by:
Press:
1. Product ID
2. Product Name
Choose an option:  2
Enter the Product Name:  Underwear 49

Search Results:
+--------------+----------------+---------+--------+---------+
| Product ID   | Product Name   |   Price | Size   |   Stock |
+==============+================+=========+========+=========+
| U3956        | Underwear 49   |     561 | XL     |      10 |
+--------------+----------------+---------+--------+---------+
```

These screenshots from the program highlight some important features. The system is designed to treat users and admin as independent without interference. The menu system is structured in a way to accommodate both customer and admin perspectives making navigation effective. Admin options clearly displaying inventory management functions while customer's option are adding/removing products and proceeding to checkout. The dynamic capabilities integrated into the system allow inventory levels to be adjusted in real time after each action. In this way users can see what items are available and with what stock. Finally, outputs in checkout, displaying item details, total cost and discounts (if applicable), show the user-friendly interface. Not to mention the notifications for invalid inputs ensuring smooth interaction.

5.2 Discussion

During this e-commerce system, the most important challenges faced during implementation was the Menu navigation and adapting different menus to suit different roles based on which method call it every time. Moreover, real-time inventory update from user's or admin's usage was a significant challenge which was faced by creating a dynamic inventory control. Last but not least, the efficient product search, in which I had to think what is more effective both for user's functionality and user's experience. While binary searching algorithm offer faster performance because of its O(log n) complexity, linear was chosen as the product_catalog is neither huge nor sorted.

5.3 Learnings

Through this project, I gained an understanding of OOP principles to solve complex problems. I explored how classes work and how to link all of them. Implementing Shopping_Cart class and its relationship with the customer, helped me better grasp how to design systems with clear and reusable components. Lastly, I also gained practical experience with dynamic data handling/updating.

6. Conclusion and Future work

On my journey of this project, I accomplished several key objectives, creating an e-commerce management system simple and effective. While it meets the core objectives, there are some areas which can be improved such us advanced search capabilities and sorting algorithms. This could be accomplished by adding advanced filters eg. By price range, ratings for better product discovery. A sorting algorithm also can be optimized to help users display the catalog by ascending/descending order. Finally, a major area for future development is the addition of a database to track orders corresponding to each customer. Currently, the system uses the same customer_id and order_id for each session.

7. Learning Objectives

Below my learning objectives will be analyzed and whether these objectives were met providing insights into my overall learning journey.

One of the primary learning objectives was to dive deeper into OOP principles. Starting with **polymorphism** in the *Menu* class and in its subclasses (*Order, Shopping_Cart, Display, Product*), I achieved a consistent interface while allowing for unique behaviors by defining the show_menu method in abstract Menu class.

This allowed me successfully to override this method in each subclass in order to tailor their specific functionality. Continuing with **inheritance**, subclasses were effectively built from abstract base classes. Order and Shopping_Cart inherited functionalities from Menu, ensuring that the main functionalities could be reused and extended across different classes without duplication. **Encapsulation** was implemented as well while Product, Shopping_Cart and Order, encapsulated data and methods related to specific operations. In this way the data was kept secure and only accessible through controlled interfaces. In addition, I also learned how to use **abstraction** and I was able to create a clear blueprint for subclasses ensuring consistency while keeping implementation details flexible.

Moreover, I learned how **searching algorithms** work and how to choose the most suitable one based on systems requirements and available sources. For instance, binary search would be much better in handling large datasets as it offers faster performance due to O(logn) complecity. However, despite its benefits, I decided to integrate linear search ensured simplicity without compromising the performance required for this specific system.

Lastly, this project assisted me generally, to understand the importance of anticipating potential failures where system can successfully recover from them, and specifically, to better understand error-handling through **try-except** blocks. This method has been applied a lot in the assignment as it allows the system to manage exceptions effectively without raising errors and stop its execution. This adds a smooth operation even when unexpected or invalid inputs are provided by the user. For example, when users attempt to add or update a product with incorrect data types or access non-existent items in the catalog, the **try-except** blocks intercept these errors, displaying clear error messages without crashing the application.

References

abc — Abstract Base Classes. (n.d.). Python Documentation. https://docs.python.org/3/library/abc.html

Bansal, I. (2023, June 8). Python tabulate module: How to Easily Create Tables in Python? - AskPython. AskPython. https://www.askpython.com/python-modules/tabulate-tables-in-python

draw.io - free flowchart maker and diagrams online. (n.d.-c). https://app.diagrams.net/